LEARNING TYPESCRIPT → NJIRANI BUILD

A 5-Week Skill Bridge for Backend Development

Goal: Build TypeScript muscle before touching Njirani
Duration: 5 weeks (part-time, alongside attachment)
Output: Express + Prisma + Zod API that persists data
Gate: No Njirani until you build a Todo API without doc lookup
Rule: One commit per day. Green GitHub or no sleep.

Prepared July 2026

Table of Contents

1. The Problem: Why You're Stuck

You have a senior-level project plan (microservices, Prisma, Docker, M-Pesa, AI) but you're still learning TypeScript syntax. That's like being handed blueprints for a skyscraper when you're figuring out how to pour concrete.

The result: you open your editor, stare at the blank file, and feel stupid. So you close it and do something else — anything else — that feels achievable.

That's not weakness. That's a human response to overwhelming scope. But it still kills your progress.

The Skill Gap

You Have	You Need
A plan for Njirani	TypeScript syntax muscle memory
Folder structure knowledge	Ability to write a function with typed parameters
Understanding of Prisma concepts	Experience running a migration and querying data
Awareness of Zod	Hands-on validation that rejects bad input

2. The Fix: Strip Everything Down

Forget Njirani for 2 weeks. Forget Prisma. Forget Docker. Forget M-Pesa. You need to build TypeScript muscle first, or every file you write will feel like guesswork.

The New Rule

> **No Njirani until you can build a Todo API with Express + Prisma + Zod without looking at docs for basic syntax.**

That's your gate. Not cruelty — protection. Building Njirani before this is like writing a novel while learning the alphabet. Possible, but painfully slow and demoralizing.

3. Week 1: TypeScript Fundamentals

No frameworks. No databases. Just TypeScript running in Node.js. Build small CLI tools that do one thing well.

Day 1–2: CLI Calculator

What to build:

A calculator that takes two numbers and an operator from the command line.

```
npx ts-node calculator.ts 5 + 3 # Output: 8
```

What you'll learn:

- number, string, boolean types
- Function parameters with types
- process.argv for CLI input
- if/else with type guards

Resources: TypeScript Handbook — Basic Types

Day 3–4: Contact Book

What to build:

A contact book. Add, list, search contacts. Store in a JSON file.

```
npx ts-node contacts.ts add "Enock" "enock@email.com" npx ts-node contacts.ts list npx ts-node contacts.ts search "Enock"
```

What you'll learn:

- interface Contact { name: string; email: string }
- Arrays: Contact[]
- Array methods: .find(), .filter(), .push()
- Reading/writing JSON files with fs

Day 5–6: Weather CLI

What to build:

Fetch weather data from a free API (OpenWeatherMap or similar).

```
npx ts-node weather.ts "Nairobi" # Output: Nairobi: 24°C, Sunny
```

What you'll learn:

- fetch() with TypeScript
- Promise, async/await
- Error handling with try/catch
- .env files with dotenv

Day 7: File-based Todo List

What to build:

A todo app that stores tasks in a JSON file. Full CRUD.

```
npx ts-node todo.ts add "Buy water" npx ts-node todo.ts list npx ts-node todo.ts done 1
npx ts-node todo.ts delete 1
```

What you'll learn:

- CRUD operations (Create, Read, Update, Delete)
- File I/O with fs.promises
- Generating IDs
- Basic CLI argument parsing

4. Week 2: Express + TypeScript

One file at a time. No database yet. Just an HTTP server that responds to requests.

Day 8–9: Hello Server

What to build:

One file: server.ts. One route: GET /health

```
import express from 'express'; const app = express(); app.use(express.json());
app.get('/health', (req, res) => { res.json({ status: 'ok' }); }); app.listen(3000, () =>
console.log('Server on port 3000'));
```

What you'll learn:

- import syntax
- Express basics
- TypeScript with Express (Request, Response types)
- npm install, tsconfig.json

Test it: curl localhost:3000/health

Day 10–11: In-Memory CRUD

What to build:

User routes with an array as 'database.' Restart = data gone. That's fine.

```
GET /users → list all users GET /users/:id → get one user POST /users → create user
PATCH /users/:id → update user DELETE /users/:id → delete user
```

What you'll learn:

- Route handlers
- URL parameters (req.params.id)
- Request body (req.body)
- Status codes (200, 201, 404)
- Array manipulation as database

Day 12–13: Zod Validation

What to build:

Same user routes, but reject bad data before it touches your logic.

```
import { z } from 'zod'; const userSchema = z.object({ name: z.string().min(2), email:
z.string().email(), age: z.number().min(0).optional() });
```

What you'll learn:

- Zod schemas
- Type inference: type User = z.infer
- Middleware pattern
- Error formatting

Day 14: Express Todo API

What to build:

The Week 1 todo app, but as a REST API. Hit it with Postman or fetch().

```
GET /todos POST /todos { title: string } PATCH /todos/:id { done: boolean } DELETE
/todos/:id
```

Still in-memory. Still one file. But now it's an API.

5. Week 3: Real Database

Persist data. Learn Prisma. Understand migrations.

Day 15–17: SQLite + Prisma

What to build:

Same todo API, but data survives server restarts.

Steps:

- npm install prisma @prisma/client
- npx prisma init
- Define Todo model in schema.prisma
- npx prisma migrate dev
- Replace array with prisma.todo.create(), findMany(), etc.

What you'll learn:

- Prisma schema syntax
- Migrations
- CRUD with Prisma Client
- Environment variables for database URL

Day 18–21: Module Refactor

What to build:

Split your one-file server into the Njirani folder structure.

```
src/ ■■■ index.ts ← starts server ■■■ app.ts ← Express setup ■■■ config/ ■ ■■■ database.ts ← Prisma client ■■■ modules/ ■ ■■■ todos/ ■ ■■■ todos.router.ts ■ ■■■ todos.controller.ts ■ ■■■ todos.service.ts ■ ■■■ todos.schema.ts ■■■ middleware/ ■■■ error.middleware.ts
```

What you'll learn:

- Why separation matters (you felt this pain in the one-file version)
- How router → controller → service flows
- Where validation lives
- How to import/export between files

6. Week 4: Back to Njirani

You now have:

- ■ TypeScript muscle memory
- ■ Express patterns internalized
- ■ Prisma working
- ■ Module structure that makes sense

Week 4 task: Port your todo API structure to Njirani. Replace Todo with User. Replace User with Estate. Same patterns, different names.

> You are no longer guessing. You are applying what you know.

7. Daily Tactics: How to Actually Do This

Problem	Solution
"I do not know what to type"	Start with the resource, type exactly what they show, then modify one thing
"I made an error and do not understand it"	Paste the error into Google. Read 3 results. If still stuck, ask with the code AND the error
"I feel like I am copying, not learning"	You are building muscle memory. Pianists play scales. You type patterns. Understanding comes from repetition + variation
"I skipped 2 days and feel guilty"	Resume at Day 1 of whatever week you are in. Do not catch up. Just continue. Consistency beats intensity

The Accountability System

- **Daily commit** — Even if it is a typo fix. Green GitHub squares are visible proof of consistency.
- **Timer** — 25 minutes coding, 5 minutes break. No phone. No Discord. No 'let me check this one thing.'
- **Public declaration** — Tweet or post: 'Building Njirani. Daily commits. No excuses.' Social pressure works.
- **Weekly review** — Every Sunday, check: Did I hit my targets? If not, why? Adjust. Do not pretend it did not happen.

8. The Gate: When You're Ready

You are ready for Njirani when you can do ALL of these without opening documentation:

- ■ Write a function with typed parameters and return type
- ■ Define an interface and use it in an array
- ■ Set up Express with one route that reads req.body
- ■ Validate req.body with Zod and return a 400 error for bad input
- ■ Run prisma migrate dev and query data with Prisma Client
- ■ Split code into router / controller / service files
- ■ Handle async errors without the server crashing

Until then, every Njirani file you write will feel like guesswork. Build the foundation first.

9. Resources

Topic	Resource
TypeScript basics	typescriptlang.org/docs/handbook/basic-types.html
TypeScript deep dive	totaltypescript.com (free tutorials)
Express.js	expressjs.com/en/starter/hello-world.html
Zod validation	zod.dev
Prisma ORM	prisma.io/docs/getting-started
Node.js best practices	github.com/goldbergyoni/nodebestpractices

> Your first commit in 17 days should be feat: add CLI calculator. That is fine. That is honest. That is progress.

— Tukae kazi.